```dart
// lib/utils/stock_movements_pdf_exporter.dart

import 'package:flutter/services.dart';
import 'package:pdf/pdf.dart';
import 'package:pdf/widgets.dart' as pw;
import 'package:easy_localization/easy_localization.dart';
import 'package:puresip_purchasing/debug_helper.dart';

class StockMovementsPdfExporter {
  static pw.Font? _cachedArabicFont;
  static pw.Font? _cachedLatinFont;

  // تحميل الخط العربي
  static Future<pw.Font> _getArabicFont() async {
    if (_cachedArabicFont != null) return _cachedArabicFont!;
    try {
      final ByteData fontData = await
rootBundle.load('assets/fonts/Cairo-Regular.ttf');
      _cachedArabicFont = pw.Font.ttf(fontData);
      safeDebugPrint('☑ Arabic font loaded successfully');
      return _cachedArabicFont!;
    } catch (e) {
      safeDebugPrint('❌ Failed to load Arabic font: $e');
      _cachedArabicFont = pw.Font.helvetica();
      return _cachedArabicFont!;
    }
  }

  // تحميل خط الأرقام والنصوص اللاتينية (لتجنب مشاكل المعاينة)
  static pw.Font _getLatinFont() {
    _cachedLatinFont ??= pw.Font.helvetica();
    return _cachedLatinFont!;
  }

  static Future<pw.Document> generateStockMovementsPdf({
    required Map<String, dynamic> data,
    required bool isArabic,
  }) async {
    final pdf = pw.Document();
    final font = await _getArabicFont();
    final latinFont = _getLatinFont();

    final companyName = data['companyName'] ?? '';
    final factoryName = data['factoryName'] ?? '';
    final startDate = data['startDate'] as DateTime?;
    final endDate = data['endDate'] as DateTime?;
    final items = data['items'] as List<dynamic>;

    pdf.addPage(
      pw.MultiPage(
        pageFormat: PdfPageFormat.a4,
        margin: const pw.EdgeInsets.all(20),
        textDirection: isArabic ? pw.TextDirection.rtl : pw.TextDirection.ltr,
        header: (context) => pw.Container(
```

```
          alignment: isArabic ? pw.Alignment.centerLeft :
pw.Alignment.centerRight,
          margin: const pw.EdgeInsets.only(bottom: 10),
          child: pw.Text(
            'stock_movements_report'.tr(),
            style: pw.TextStyle(font: font, fontSize: 9, color:
PdfColors.grey500),
          ),
        ),
        footer: (context) => pw.Container(
          alignment: pw.Alignment.center,
          margin: const pw.EdgeInsets.only(top: 15),
          child: pw.Text(
            '${context.pageNumber} / ${context.pagesCount}',
            style: pw.TextStyle(font: latinFont, fontSize: 9, color:
PdfColors.grey600),
          ),
        ),
        build: (context) {
          return [
            // عنوان التقرير
            pw.Center(
              child: pw.Text(
                'stock_movements_report'.tr(),
                style: pw.TextStyle(font: font, fontSize: 22, fontWeight:
pw.FontWeight.bold),
              ),
            ),
            pw.SizedBox(height: 20),

            // معلومات الشركة والمصنع
            pw.Text('${'company'.tr()}: $companyName',
              style: pw.TextStyle(font: font, fontSize: 12)),
            pw.Text('${'factory'.tr()}: $factoryName',
              style: pw.TextStyle(font: font, fontSize: 12)),
            if (startDate != null && endDate != null)
              pw.Text('${'period'.tr()}: ${_formatDate(startDate)} ${'to'.tr()}
${_formatDate(endDate)}',
                style: pw.TextStyle(font: font, fontSize: 12)),
            pw.SizedBox(height: 20),
            pw.Divider(thickness: 0.5, color: PdfColors.grey400),
            pw.SizedBox(height: 10),

            // الأصناف المتنقلة
            ...items.map((item) => _buildItemSection(Map<String,
dynamic>.from(item), isArabic, font, latinFont)),
          ];
        },
      ),
    );

    return pdf;
  }
```

```
  static pw.Widget _buildItemSection(Map<String, dynamic> item, bool isArabic,
pw.Font font, pw.Font latinFont) {
    final itemName = item['name'] ?? '';
    final itemCategory = item['category'] ?? 'raw_material';
    final openingBalance = (item['openingBalance'] ?? 0).toDouble();
    final closingBalance = (item['closingBalance'] ?? 0).toDouble();
    final currentStock = (item['currentStock'] ?? 0).toDouble();
    final movements = item['movements'] as List<dynamic>? ?? [];

    final categoryText = isArabic
        ? (itemCategory == 'raw_material' ? 'مواد خام' : 'مواد تعبئة وتغليف')
        : (itemCategory == 'raw_material' ? 'Raw Material' : 'Packaging
Material');

    return pw.Container(
      margin: const pw.EdgeInsets.only(bottom: 15),
      child: pw.Column(
        crossAxisAlignment: pw.CrossAxisAlignment.start,
        children: [
          // رأس الصنف مع الفئة
          pw.Container(
            padding: const pw.EdgeInsets.all(8),
            decoration: const pw.BoxDecoration(
              color: PdfColors.grey300,
              borderRadius: pw.BorderRadius.all(pw.Radius.circular(4)),
            ),
            child: pw.Row(
              mainAxisAlignment: pw.MainAxisAlignment.spaceBetween,
              children: [
                pw.Expanded(
                  child: pw.Text('${'product'.tr()}: $itemName ($categoryText)',
                    style: pw.TextStyle(font: font, fontWeight:
pw.FontWeight.bold, fontSize: 11)),
                ),
                pw.Row(
                  children: [
                    _buildBalanceText('opening_balance'.tr(), openingBalance,
PdfColors.blue, font, latinFont),
                    pw.SizedBox(width: 10),
                    _buildBalanceText('closing_balance'.tr(), closingBalance,
PdfColors.green, font, latinFont),
                    pw.SizedBox(width: 10),
                    _buildBalanceText('current_stock'.tr(), currentStock,
PdfColors.orange, font, latinFont),
                  ],
                ),
              ],
            ),
          ),
          pw.SizedBox(height: 8),

          // جدول الحركات
          _buildMovementsTable(List<Map<String, dynamic>>.from(movements),
isArabic, font, latinFont),
```

```
        ],
      ),
    );
  }

  static pw.Widget _buildBalanceText(String label, double value, PdfColor color,
pw.Font font, pw.Font latinFont) {
    return pw.RichText(
      text: pw.TextSpan(
        children: [
          pw.TextSpan(text: '$label: ', style: pw.TextStyle(font: font, color:
color, fontSize: 9)),
          pw.TextSpan(text: value.toStringAsFixed(2), style: pw.TextStyle(font:
latinFont, color: color, fontSize: 9)),
        ],
      ),
    );
  }

  static pw.Widget _buildMovementsTable(
    List<Map<String, dynamic>> movements,
    bool isArabic,
    pw.Font font,
    pw.Font latinFont
  ) {
    if (movements.isEmpty) {
      return pw.Container(
        padding: const pw.EdgeInsets.all(10),
        alignment: pw.Alignment.center,
        child: pw.Text('no_movements'.tr(),
          style: pw.TextStyle(font: font, color: PdfColors.grey600)),
      );
    }

    Map<int, pw.TableColumnWidth> columnWidths = {
      0: const pw.FixedColumnWidth(30),   // #
      1: const pw.FixedColumnWidth(85),   // التاريخ
      2: const pw.FixedColumnWidth(110),  // نوع الحركة
      3: const pw.FixedColumnWidth(70),   // وارد
      4: const pw.FixedColumnWidth(70),   // منصرف
      5: const pw.FixedColumnWidth(85),   // الرصيد
    };

    List<String> headers = ['#', 'Date', 'Type', 'In', 'Out', 'Balance'];

    if (isArabic) {
      headers = ['balance', 'outgoing', 'incoming', 'movement_type', 'date', '#']
          .map((h) => h.tr()).toList();

      columnWidths = {
        0: const pw.FixedColumnWidth(85),   // الرصيد
        1: const pw.FixedColumnWidth(70),   // منصرف
        2: const pw.FixedColumnWidth(70),   // وارد
        3: const pw.FixedColumnWidth(110),  // نوع الحركة
```

```
        4: const pw.FixedColumnWidth(85),   // التاريخ
        5: const pw.FixedColumnWidth(30),   // #
      };
    }

    final List<pw.TableRow> tableRows = [];

    tableRows.add(
      pw.TableRow(
        decoration: const pw.BoxDecoration(color: PdfColors.grey200),
        children: headers.map((header) => pw.Padding(
          padding: const pw.EdgeInsets.all(6),
          child: pw.Text(
            header,
            textAlign: pw.TextAlign.center,
            style: pw.TextStyle(font: font, fontWeight: pw.FontWeight.bold,
fontSize: 9),
          ),
        )).toList(),
      ),
    );

    for (int i = 0; i < movements.length; i++) {
      final m = movements[i];

      final cellIndex = _buildCell('${i + 1}', latinFont);
      final cellDate = _buildCell(_formatDate(m['date']), latinFont);
      final cellType = _buildCell(m['type_text'] ?? '', font);
      final cellIn = _buildCell((m['in'] ?? 0) > 0 ? (m['in'] as
num).toStringAsFixed(2) : '-', latinFont);
      final cellOut = _buildCell((m['out'] ?? 0) > 0 ? (m['out'] as
num).toStringAsFixed(2) : '-', latinFont);
      final cellBalance = _buildCell((m['balance'] ??
0).toDouble().toStringAsFixed(2), latinFont);

      List<pw.Widget> rowCells;

      if (isArabic) {
        rowCells = [cellBalance, cellOut, cellIn, cellType, cellDate, cellIndex];
      } else {
        rowCells = [cellIndex, cellDate, cellType, cellIn, cellOut, cellBalance];
      }

      tableRows.add(
        pw.TableRow(children: rowCells),
      );
    }

    return pw.Table(
      border: pw.TableBorder.all(color: PdfColors.grey400, width: 0.5),
      columnWidths: columnWidths,
      children: tableRows,
    );
  }
```

```
  static pw.Widget _buildCell(String text, pw.Font font) {
    return pw.Padding(
      padding: const pw.EdgeInsets.all(5),
      child: pw.Text(
        text,
        textAlign: pw.TextAlign.center,
        style: pw.TextStyle(font: font, fontSize: 9),
      ),
    );
  }

  static String _formatDate(dynamic date) {
    if (date == null) return '';
    final DateTime dateTime = date is DateTime ? date :
DateTime.parse(date.toString());
    return '${dateTime.year}/${dateTime.month.toString().padLeft(2,
'0')}/${dateTime.day.toString().padLeft(2, '0')}';
  }
}
```